

# volatile solutions

---

Project: ISO JTC1/SC22/WG21: Programming Language C++

Audience: Core Working Group

Document: P0278r0

Date: 2016-02-14 ♥

Authors: Jon Kalb <jon at kalbweb dot com>

Dan Saks <dan at dansaks dot com>

## I. Introduction

This proposal deals with three issues related to objects declared with the `volatile` qualifier.

The goals are for the standard to say:

0. that the behavior of reading from an uninitialized volatile object is not undefined.
1. that the implementation can never assume the value of a volatile object, even if it has just written it.
2. that objects qualified with `const volatile` need not be initialized.

## II. Background

The common use case for objects to be declared with the `volatile` qualifier is to support memory-mapped I/O. Such ports look to the software as if they are memory at a particular address. This design has a number of implications including:

- any access, even a write of the existing value or a read, can have side effects
- the address may not “hold” a value written to it, that is, the read value of port may be unrelated to the value written, even if it was “just” written
- some ports, and therefore some objects declared with the `volatile` qualifier, may be read-only and some may be write-only

Because “regular” memory doesn’t have these properties and we want to allow the implementation to optimize for the common case rather than uncommon case of memory-mapped I/O, we allow the implementation to make assumptions about memory that are true for regular memory, but not true for ports.

To support memory-mapped I/O we declare such objects with the `volatile` qualifier. The implications of this are spelled out in 1.9/8 and 7.1.6.1/7 (of [n4567](#)), which essentially give the programmer greater control of exactly how the volatile object is accessed and forbid certain optimizations that make sense (only) for regular memory.

## III. Motivation and Scope

### Use Case 0: Reading from an uninitialized volatile object

Consider a 32-bit read-only port located at location 0x3FF6000 declared and used like this:

```
using reg_t = uint32_t volatile;

reg_t* readonly_reg{new ((void*)0x3FF6000) reg_t};

uint32_t current_value{*readonly_reg};
```

In Use Case 0 we have a read from a value (pointed to by `readonly_reg`) that has not been initialized. Since this is a read-only port, this is correct (and the only permissible) usage. However, according to 8.5/12 reading an uninitialized value is undefined behavior. In current practice, implementations simply perform the read, but a conforming implementation could optimize away any code path on which this happens. *We assert that reading from an uninitialized volatile object should be defined behavior.*

### Use Case 1: Assuming a written value is "held" by a volatile object

Consider a byte sized port that is read-write, but the read value is implemented to be the number of bytes that have been written to the port (or some other value that, from the implementation's point of view, is unrelated to the value written).

#### Use Case 1a

```

extern char volatile IOPort;

char frob(char arg)
{
    IOPort = arg;
    IOPort = 'a';
    return IOPort;
}

char value_read = frob('b');

```

### Use Case 1b

```
char value_read = IOPort = 'a';
```

In Use Cases 1a and 1b, the user expects the value of `value_read` to be a value actually read from `IOPort`, but an implementation is allowed to take advantage of the fact that it knows the value of `IOPort` because it just set it and use 'a' instead of actually reading the port. [Note: A read of the volatile object is required because that is expected of the abstract machine, but the implementation is able to anticipate and use the value before that load is complete.]

In this case the assumption is incorrect because the port's read value is not (necessarily) equal to the last value written to it. (Actually, it isn't clear whether or not an implementation is free to do this. It doesn't seem to be in the spirit of volatile, but it doesn't seem to be forbidden either.) *We assert that the standard should specifically forbid the implementation from making assumptions about the read value of volatile objects.*

### Use Case 2: Making `const volatile` read-only

Again consider the 32-bit read-only port located at location 0x3FF6000. Because it is read-only, the programmer declares it `const` so that the implementation will catch writes to the port at compile time, because they are now non-conforming.

```
using ro_reg_t = uint32_t volatile const;

ro_reg_t* readonly_reg{new ((void*)0x3FF6000) ro_reg_t}; // Error: No initialization of const volatile object.
```

This fails to compile because the value created at the port is declared `const`, but not given an initializer. But as a read-only port, no initialization is permissible.

Due to the nature of volatile objects, the implementation should never assume that they are not initialized, nor should it insist that every volatile object be written to. *We assert that the standard should not require that objects declared `const volatile` be initialized.*

## IV. Compatibility

Requiring implementations to treat reads from uninitialized volatile objects as defined behavior, probably results in no changes to existing practice.

Requiring implementations to read from volatile objects instead of assume they hold the value assigned them, may “pessimize” some existing code, but for any existing code that yields a different result, the new results will be the expected behavior.

Allowing objects declared `const volatile` to be uninitialized will break no existing code, but will require changes to existing implementations.

## V. Technical Specifications

In the last sentence of 8.5/7 specify that the requirement doesn't apply to volatile types:

If a program calls for the default initialization of an object of a [non-volatile](#) const-qualified type T, T shall be a class type with a user-provided default constructor.

The intention is that `const volatile` objects do not need to be initialed. Note: There may also be a required change for non-class type objects.

In 8.5/12 (add a new bullet before existing bullet 8.5/12.1):

— If an indeterminate value is produced by the evaluation of an uninitialized volatile object, then the result of the

operation is an indeterminate value.

The intention is that reading from an uninitialized value is not undefined behavior.

In 3.9.3 add paragraph 7:

The value stored within a volatile object can change between accesses, and it is unspecified whether and when this happens and whether such changes can contribute to a data race.

The intention is that the implementation can never assume the read value of volatile.

## **VI. Acknowledgements**

The authors want to thank Richard Smith for the early feedback and guidance.